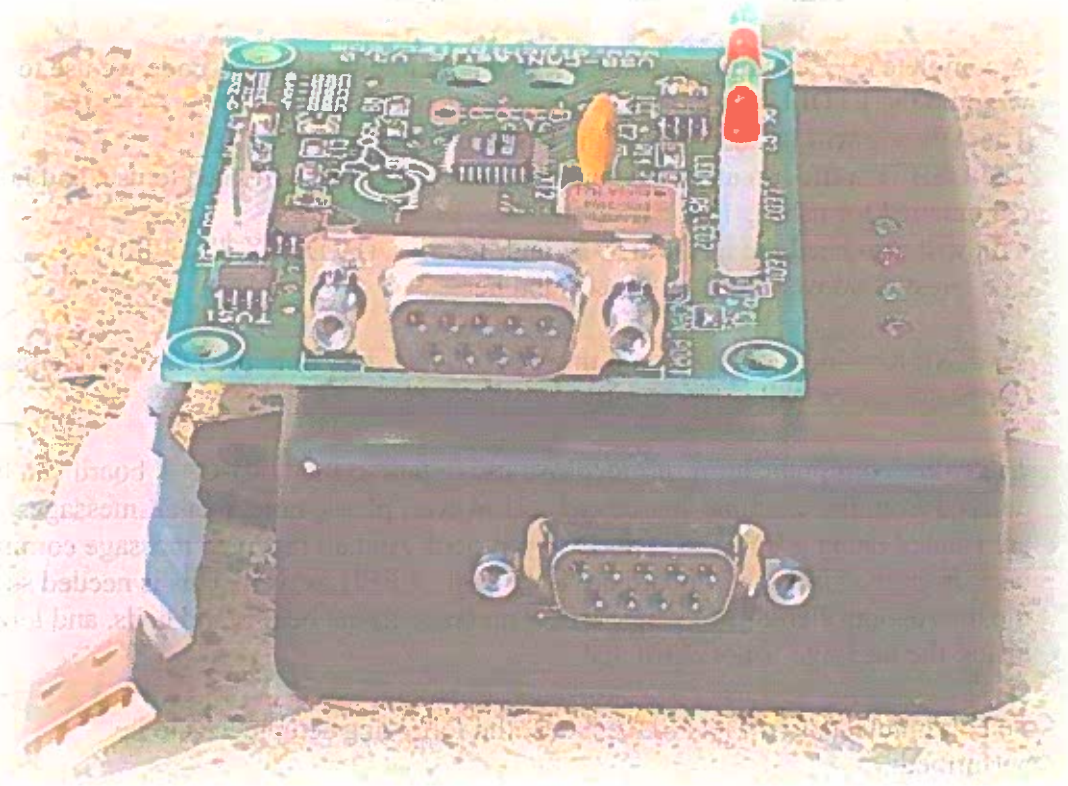
Developers Quick-Start Guide for the USB-CAN V3.0 API

Apox Controls 2004



1. API Overview

The USB-CAN API gives the developer a very simple mechanism to attach to multiple USB-CAN boards. The software engineer needs no knowledge of the USB bus to get started. All USB drivers are plug and play.

The software engineer needs only to be concerned with the CAN protocol they intend to receive and transmit. There are many additional functions to which you can access the microcontrollers' hardware directly, but these functions are typically not needed to send and receive messages.

A complete C++ example software is given, including all of the code we use to wrap up the FTDI USB libraries.
The API you will use consists of the USBFT.cpp and USBFT.h.
The USBFT.h file has all of the function prototypes you will need to use, and is commented for usage.
You will also need the FTDI drivers, and libraries. (FTD2XX.dll,.h,.lib)
The latest windows drivers can be found here.
http://www.ftdichip.com/FTD2XXDriver.htm
WARNING: You DO NOT want the VCP drivers. Use only the D2XX drivers! (Or performance will suffer greatly)

The entire communications protocol we use to talk to the USB-CAN board can be derived from the example source code. However, please note, that all messages are framed using a BYTE STUFFING protocol. And all received message coming back from the USB-CAN board are BYTE STUFFED as well. This is needed so the microcontroller knows where a new message frame begins and ends, and too insure the message is not corrupted.

To communicate to a CAN Slave Node, the following setup needs to be performed:

1.) **Find what USB-CAN boards exist on the BUS.**
Using the function

```
CUsbFt:: ListUSBCANBoards(&numDevs, &buf);
```

This function will return the number of USB-CAN devices found, as well as a list of all of the USB-CAN boards serial numbers. Even though the USB-CAN boards share a common Vendor ID, and Product ID, they are all programmed with a different serial number so that multiple USB-CAN boards can be attached to one USB bus.

2.) Open the USB-CAN boards:
Instantiate a CUsbFt object and passing in the serial number string (obtained from step 1). Using the constructor.

```
pUsbFt = new CUsbFt(serialNumString);.
```

NOTE: You can give the serial number an Alias name in the Windows registry if you desire using the following functions:

```
BOOL CUsbFt::UpdateSerialNumberToName(CString mapping)
CString CUsbFt::MapSerialNumberToName(char *serialNumber)
```

3.) Initializing a USB-CAN board or reestablishing a USB connection:
Upon the start of your program, or If you lost connection to a USB device, for example, user unplugged the board, and plugged it back in, but you do not want to restart your software, you call

```
CUsbFt:: InitUSBDevice()
```

```
If you want Information about the USB device, you can read the
USB chips EEPROM.
CUsbFt::ReadFT245_Data(ManufacturerBuf,ManufacturerIdBuf,
                       DescriptionBuf,SerialNumberBuf);
```

4.) Updating the FIRMWARE on the board:
If you need to update the latest firmware to the board, you can do so by calling the function

```
DWORD CUsbFt::StartDownloadUSB( char *filename )
```

The filename would contain the path of the .HEX file with the latest firmware.
Note: This should only be done with a new firmware release, not everytime the board is started.

5.) Define Callbacks:
Polling for new messages is a typically unacceptable way to receive Asynchronous CAN messages from the USB-CAN board. So a Callback function will need to be setup. Upon successful completion of a received message. Your callback function will be called to indicate that a message is waiting in the Queue. Remember your callback function will be executed from a different thread, So in a Windows program, your program should then send a Windows message to your GUI for a screen update. (There are two callbacks, One for normal messages coming back from the USB-CAN board, and one for EMERGENCY messages for and problems that have occurred.)

```
void CCanTxRx::Init(CUsbFt *pusbft)
{
m_pUsbFt = pusbft;
m_pUsbFt->DefineCANMsgCallback(CANDataCallback, this);
m_pUsbFt->DefineUSBEmergencyCallback(USBEmergencyCallback, this);
}
```

6.) Switch the USB-CAN board to MAIN CODE!!!!
WARNING: The USB-CAN board always boots up into BOOTCODE. BOOTCODE allows the board to be updated with the latest firmware. If a green LED is blinking a steady pattern, the board is in BOOTCODE. In order for you to send and receive messages you must tell the bootcode to execute the main firmware.

To find out which code is running, you can query the board with the following function

```
DWORD    AskWhichCodeIsRunning(bool *bMainRunning);

You can switch back and forth using the following functions
void  SwitchToBootCode();
void  SwitchToMainCode();

Once the board is running from main firmware, you can begin
setting up the CAN modes you would like.
```

7.) Setup the CAN BAUD rate:

Currently 125K,250K,500K, and 1Mbps are selectable by the user. (If your Slave node does not match your settings, you will not be able to communicate)

```
SetUsbCanBaudRate( SET_BAUD_1MEG );
```

8.) Setup your Receive Masks and Buffers.

If you want to receive all messages (As a Master Node would typically want to do), you set all buffers and filters to 0x00000000.

Use the following functions to read and write the CAN filters.

```
DWORD RequestFilterUpdate(USB_CAN_FILTERS *pUSBCANFilter);
DWORD WriteFilterData(USB_CAN_FILTERS *pUSBCANFilter);
```

If you want to receive Standard (11 bit) or Extended (29 bit) Frames. (or Both) There are two receive buffers, each buffer can receive either 29bit or 11 bit messages. You need to select which message type you want each buffer to receive. If you set the MSB (most significant bit) of the filter or mask value (a 32 bit number), then extended mode will be selected for that receive buffer

IMPORTANT: Receive buffer 1 has 1 Mask and 2 filter settings, however Receive Buffer 2 has 1 Mask, and 4 filter values.

9.) Change the USB-CAN to NORMAL MODE.

The USB-CAN board has multiple modes. For development purposes LOOPBACK mode is very useful, since you do not need any other CAN boards on the bus to receive CAN messages. (Just send a message, and as long as your filters are setup correctly, you will immediately receive the message back) NORMAL MODE is just as it sounds, if you want to transmit and receive CAN messages on the bus, you must switch to NORMAL MODE. If you change parameters such as BAUD rate or Message Filters, the board will switch to CONFIG MODE automatically. WARNING: You must switch back to NORMAL mode after your configuration is complete.

Use these functions to switch modes:

```
DWORD SetConfigMode(void);
DWORD SetNormalMode(void);
DWORD SetListenOnlyMode(void);
DWORD SetDisabledMode(void);
DWORD SetLoopBackMode(void);
```

10.) Receiving a CAN message:

When you get a CAN message, your CALLBACK function will be called.
You need to grab the received message out of a storage queue.

```
Example:
RxData rxData;
m_pUsbFt->ReadCANMsg(&rxData))
```

The message will have the following structure.

```
typedef struct
{
// the following is embedded into the header byte;
bool who; // this is set to 1 if message originated from the
                  USB-CAN board
bool rtr; // set to 1 if the RTR bit was
            set on the received message
bool ext; //0=11 bit identifier  1=29 bit identifier
BYTE header;
BYTE command; // The command type
DWORD Id; // either the 11 or 29 bit identifier
DWORD timestamp;
BYTE len;
BYTE data[255];
BYTE txflags;
BYTE rxflags;
}RxData;
```

11.) Transmitting a CAN message:

There are a few options the programmer needs to consider when transmitting a CAN message.
The RTR bit. (Some protocols use the RTR bit to request data from a Slave node)
The EXT bit (if set, tells program to transmit a 29 bit extended frame message, 11 bit otherwise)
The ID is either an 11 bit or 29 bit value (implemented as a 32 bit in software) depending on the EXT bit
The DATA array is 0 to 8 bytes of data.
The DATALEN field is the number of DATA bytes you want to transmit.
The following function transmits a CAN message.

```
DWORD CUsbFt:: SendRawCANMessage(bool rtr, bool ext, BYTE txFlags,
                  DWORD Id, BYTE *data, unsigned int dataLen);
```

D2XX Programmer's Guide

Introduction

If you would like to modify the D2XX driver wrapper code yourself. The following info is included for your reference.

FT_GetDeviceInfo

Description Get device information.

Syntax FT_STATUS **FT_GetDeviceInfo** (FT_HANDLE ft*Handle*, FT_DEVICE **pftType*, LPDWORD *lpdwID*, PCHAR *pcSerialNumber*, PCHAR *pcDescription*, PVOID *pvDummy*)

Parameters

ftHandle *FT*_HANDLE: handle returned from *FT_Open* or *FT_OpenEx*.
pftType Pointer to unsigned long to store device type.
lpdwId Pointer to unsigned long to store device ID.
pcSerialNumber Pointer to buffer to store device serial number as a null-terminated string.
pcDescription Pointer to buffer to store device description as a null-terminated string.
pvDummy Reserved for future use - should be set to NULL.

Return Value FT_STATUS: FT_OK if successful, otherwise the return value is an FT error code.

Remarks

This function is used to return the device type, device ID, device description and serial number.

The device ID is encoded in a DWORD - the most significant word contains the vendor ID, and the least significant word contains the product ID. So the returned ID 0x04036001 corresponds to the device ID VID_0403&PID_6001.

Example

This example shows how to get information about a device.

```
FT_HANDLE ftHandle;   // valid handle returned from FT_OpenEx
FT_DEVICE ftDevice;
FT_STATUS ftStatus;
DWORD deviceID;
```

```
char SerialNumber[16];
char Description[64];

ftStatus = FT_GetDeviceInfo(
                ftHandle,
                &ftDevice,
                &deviceID,
                SerialNumber,
                Description,
                NULL
                );
if (ftStatus == FT_OK) {
        if (ftDevice == FT_DEVICE_232BM)
                ; // device is FT232BM
        else if (ftDevice == FT_DEVICE_232AM)
                ; // device is FT8U232AM
        else if (ftDevice == FT_DEVICE_100AX)
                ; // device is FT8U100AX
        else
                ; // unknown device (this should not happen!)
        // deviceID contains encoded device ID
        // SerialNumber, Description contain 0-terminated strings
}
else {
        // FT_GetDeviceType FAILED!
}
```

FT_SetResetPipeRetryCount

Description Set the ResetPipeRetryCount.

Syntax FT_STATUS **FT_SetResetPipeRetryCount** (FT_HANDLE ft*Handle*, DWORD *dwCount*)

Parameters

ftHandle FT_HANDLE: handle returned from *FT_Open* or *FT_OpenEx*.
dwCount DWORD: unsigned long containing required ResetPipeRetryCount.

Return Value FT_STATUS: FT_OK if successful, otherwise the return value is an FT error code.

Remarks

This function is used to set the ResetPipeRetryCount. ResetPipeRetryCount controls the maximum number of times that the driver tries to reset a pipe on which an error has occurred. ResetPipeRequestRetryCount defaults to 50. It may be necessary to increase this value in noisy environments where a lot of USB errors occur.

Example

This example shows how to set the ResetPipeRetryCount to 100.

```
FT_HANDLE ftHandle;    // valid handle returned from FT_OpenEx
FT_STATUS ftStatus;
DWORD dwRetryCount;

dwRetryCount = 100;
ftStatus = FT_SetResetPipeRetryCount(ftHandle,dwRetryCount);
if (ftStatus == FT_OK) {
        // ResetPipeRetryCount set to 100
```

```
}
else {
        // FT_SetResetPipeRetryCount FAILED!
}
```

FT_StopInTask

Description Stops the driver's IN task.

Syntax FT_STATUS **FT_StopInTask** (FT_HANDLE ft*Handle*)

Parameters

ftHandle FT_HANDLE: handle returned from *FT_Open* or *FT_OpenEx*.

Return Value FT_STATUS: FT_OK if successful, otherwise the return value is an FT error code.

Remarks

This function is used to put the driver's IN task (read) into a wait state. It can be used in situations where data is being received continuously, so that the device can be purged without more data being received. It is used together with FT_RestartInTask which sets the IN task running again.

Example

This example shows how to use FT_StopInTask.

```
FT_HANDLE ftHandle;    // valid handle returned from FT_OpenEx
FT_STATUS ftStatus;

do {
        ftStatus = FT_StopInTask(ftHandle);
} while (ftStatus != FT_OK);

//
// Do something - for example purge device
//

do {
        ftStatus = FT_RestartInTask(ftHandle);
} while (ftStatus != FT_OK);
```

FT_RestartInTask

Description Restart the driver's IN task.

Syntax FT_STATUS **FT_RestartInTask** (FT_HANDLE ft*Handle*)

Parameters

ftHandle FT_HANDLE: handle returned from *FT_Open* or *FT_OpenEx*.

Return Value FT_STATUS: FT_OK if successful, otherwise the return value is an FT error code.

Remarks

This function is used to restart the driver's IN task (read) after it has been stopped by a call to FT_StopInTask.

Example

This example shows how to use FT_RestartInTask.

```
FT_HANDLE ftHandle;    // valid handle returned from FT_OpenEx
FT_STATUS ftStatus;

do {
        ftStatus = FT_StopInTask(ftHandle);
} while (ftStatus != FT_OK);

//
// Do something - for example purge device
//

do {
        ftStatus = FT_RestartInTask(ftHandle);
} while (ftStatus != FT_OK);
```

FT_ResetPort

Description Send a reset command to the port.

Syntax FT_STATUS **FT_ResetPort** (FT_HANDLE ft*Handle*)

Parameters

ftHandle FT_HANDLE: handle returned from *FT_Open* or *FT_OpenEx*.

Return Value FT_STATUS: FT_OK if successful, otherwise the return value is an FT error code.

Remarks

This function is used to attempt to recover the port after a failure.

Example

This example shows how to reset the port.

```
FT_HANDLE ftHandle;    // valid handle returned from FT_OpenEx
FT_STATUS ftStatus;

ftStatus = FT_ResetPort(ftHandle);
if (ftStatus == FT_OK) {
        // Port has been reset
}
else {
        // FT_ResetPort FAILED!
}
```